

Assignment:5B ELO Calculations

Tenzin Lhamo Thakuri

2026-10-01

Task

Chess ELO calculations

Based on difference in ratings between the chess players and each of their opponents in our Project 1 tournament, calculate each player's expected score (e.g. 4.3) and the difference from their actual score (e.g 4.0). List the five players who most overperformed relative to their expected score, and the five players that most underperformed relative to their expected score.

Approach

Step 1: Calculate the expected score of each player using the ELO formula mentioned here <https://www.youtube.com/watch?v=AsYfbmp0To0> Step 2: calculate difference of players expected score from their actual score, in my data "Total_Points". Step 3: Sort result data in descending order to find five most overperformed players relative to their actual score. step 4: Sort result data in ascending order to find five most underperformed players relative to their actual score.

Anticipated Challenges

-Find expected score of all the palyers, sort them to analyse it. ## Code Base

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.0      v readr      2.2.0
## v forcats    1.0.1      v stringr    1.6.0
## v ggplot2    4.0.3      v tibble     3.3.1
## v lubridate  1.9.5      v tidyr      1.3.2
## v purrr      1.2.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

# 1. Read and clean text
text <- read_lines("https://raw.githubusercontent.com/lhamo07/Data-607-Assignment/refs/heads/Project1/tournamentInfo_df <- tibble(text = text) %>%
  filter(!str_detect(text, "^\\s*--+\\s*$")) %>%
```

```

slice(-(1:2)) %>%
mutate(person = ceiling(row_number() / 2)) %>%
group_by(person) %>%
summarise(text = paste(text, collapse = " "), .groups = "drop") %>%
mutate(
  Pair_Num = as.numeric(str_extract(text, "^\\s*(\\d+)")),
  Player_name = str_trim(str_extract(text, "(?<=\\|\\|\\s)[A-Z]+(?:[-\\s]+[A-Z]+)+(?=\\s*\\|\\|)")),
  Total_Points = as.numeric(str_extract(text, "(?<=\\|\\|\\s*\\d+\\.\\d+)")),
  Player_pre_Rating = as.numeric(str_extract(text, "(?<=R:)\\s*\\d+"))
)

# 2. Extract raw round text blocks and parse opponent ID numbers
rounds_df <- tournamentInfo_df %>%
  mutate(rounds_text = str_extract(text, "(\\|\\|\\s*[WLDXHP\\s]\\s*\\d*\\s*){7}(?=\\|\\|)")) %>%
  separate(rounds_text, into = c("drop", "R1", "R2", "R3", "R4", "R5", "R6", "R7"), sep = "\\|\\|") %>%
  mutate(across(R1:R7, ~ as.numeric(str_extract(.x, "\\d+")))) %>%
  select(Pair_Num, Player_name, Total_Points, Player_pre_Rating, R1:R7)

# 3. Create a clean lookup table for player ratings
rating_lookup <- rounds_df %>%
  select(Pair_Num, Player_pre_Rating)

# 4. Map opponent IDs to their respective pre-tournament ratings
opponent_info_df <- rounds_df %>%
  pivot_longer(cols = R1:R7, names_to = "Round", values_to = "Opponent_Num") %>%
  left_join(rating_lookup, by = c("Opponent_Num" = "Pair_Num")) %>%
  rename(Opponent_Pre_Rating = Player_pre_Rating.y, Player_pre_Rating = Player_pre_Rating.x) %>%

# Pivot back into a wide format showing Opponent ID and Opponent Rating side-by-side per round
pivot_wider(
  names_from = Round,
  values_from = c(Opponent_Num, Opponent_Pre_Rating),
  names_glue = "{Round}_{.value}"
) %>%
# Reorder columns chronologically (R1 info, R2 info, etc.)
select(
  Pair_Num, Player_name, Total_Points, Player_pre_Rating,
  R1_Opponent_Pre_Rating,
  R2_Opponent_Pre_Rating,
  R3_Opponent_Pre_Rating,
  R4_Opponent_Pre_Rating,
  R5_Opponent_Pre_Rating,
  R6_Opponent_Pre_Rating,
  R7_Opponent_Pre_Rating
)

opponent_info_df

```

```

## # A tibble: 64 x 11
##   Pair_Num Player_name Total_Points Player_pre_Rating R1_Opponent_Pre_Rating
##   <dbl> <chr>         <dbl>         <dbl>         <dbl>
## 1      1      1 GARY HUA           6          1794          1436
## 2      2      2 DAKSHESH DARU~           6          1553          1175

```

```
## 3      3 ADITYA BAJAJ      6      1384      1641
## 4      4 PATRICK H SCH~    5.5    1716    1363
## 5      5 HANSHI ZUO      5.5    1655    1242
## 6      6 HANSEN SONG      5      1686    1399
## 7      7 GARY DEE SWAT~   5      1649    1092
## 8      8 EZEKIEL HOUGH~   5      1641    1384
## 9      9 STEFANO LEE      5      1411    1745
## 10     10 ANVIT RAO      5      1365    1604
## # i 54 more rows
## # i 6 more variables: R2_Opponent_Pre_Rating <dbl>,
## #   R3_Opponent_Pre_Rating <dbl>, R4_Opponent_Pre_Rating <dbl>,
## #   R5_Opponent_Pre_Rating <dbl>, R6_Opponent_Pre_Rating <dbl>,
## #   R7_Opponent_Pre_Rating <dbl>
```

Calculate ELO

The standard Elo expected score formula is

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}$$

Where: * R_A is Player A's current rating. * R_B is Player B's current rating.

```
elo_df<-opponent_info_df%>%
# Drops any row where Rating_A or Rating_B is missing

drop_na(R1_Opponent_Pre_Rating, R2_Opponent_Pre_Rating, R3_Opponent_Pre_Rating,
        R4_Opponent_Pre_Rating, R5_Opponent_Pre_Rating, R6_Opponent_Pre_Rating,
        R7_Opponent_Pre_Rating)%>% mutate(ELO_R1=1/(1+10^((R1_Opponent_Pre_Rating-Player_pre_Rating)/400)),
        ELO_R2=1/(1+10^((R2_Opponent_Pre_Rating-Player_pre_Rating)/400)),
        ELO_R3=1/(1+10^((R3_Opponent_Pre_Rating-Player_pre_Rating)/400)),
        ELO_R4=1/(1+10^((R4_Opponent_Pre_Rating-Player_pre_Rating)/400)),
        ELO_R5=1/(1+10^((R5_Opponent_Pre_Rating-Player_pre_Rating)/400)),
        ELO_R6=1/(1+10^((R6_Opponent_Pre_Rating-Player_pre_Rating)/400)),
        ELO_R7=1/(1+10^((R7_Opponent_Pre_Rating-Player_pre_Rating)/400)),
        ELO_total=ELO_R1+ELO_R2+ELO_R3+ELO_R4+ELO_R5+ELO_R6+ELO_R7

)
elo_df
```

```
## # A tibble: 41 x 19
##   Pair_Num Player_name   Total_Points Player_pre_Rating R1_Opponent_Pre_Rating
##   <dbl>   <chr>           <dbl>           <dbl>           <dbl>
## 1      1      1 GARY HUA             6             1794            1436
## 2      2      2 DAKSHESH DARU~        6             1553            1175
## 3      3      3 ADITYA BAJAJ          6             1384            1641
## 4      4      4 PATRICK H SCH~        5.5            1716            1363
## 5      5      5 HANSHI ZUO             5.5            1655            1242
## 6      6      6 HANSEN SONG            5             1686            1399
## 7      7      7 GARY DEE SWAT~        5             1649            1092
## 8      8      8 EZEKIEL HOUGH~        5             1641            1384
## 9      9      9 STEFANO LEE           5             1411            1745
```

```
## 10      10 ANVIT RAO      5      1365      1604
## # i 31 more rows
## # i 14 more variables: R2_Opponent_Pre_Rating <dbl>,
## #   R3_Opponent_Pre_Rating <dbl>, R4_Opponent_Pre_Rating <dbl>,
## #   R5_Opponent_Pre_Rating <dbl>, R6_Opponent_Pre_Rating <dbl>,
## #   R7_Opponent_Pre_Rating <dbl>, ELO_R1 <dbl>, ELO_R2 <dbl>, ELO_R3 <dbl>,
## #   ELO_R4 <dbl>, ELO_R5 <dbl>, ELO_R6 <dbl>, ELO_R7 <dbl>, ELO_total <dbl>
```

Calculate difference from actual score

```
final_df<-elo_df%>%
  mutate(Score_minus_Expected=Total_Points-ELO_total)%>%
  select(Pair_Num,Player_name,Total_Points,Player_pre_Rating,ELO_total,Score_minus_Expected)

final_df
```

```
## # A tibble: 41 x 6
##   Pair_Num Player_name      Total_Points Player_pre_Rating ELO_total
##   <dbl>   <chr>          <dbl>          <dbl>      <dbl>
## 1         1 GARY HUA             6            1794      5.16
## 2         2 DAKSHESH DARURI        6            1553      3.78
## 3         3 ADITYA BAJAJ           6            1384      1.95
## 4         4 PATRICK H SCHILLING    5.5            1716      4.74
## 5         5 HANSHI ZUO             5.5            1655      4.38
## 6         6 HANSEN SONG              5            1686      4.94
## 7         7 GARY DEE SWATHELL       5            1649      4.58
## 8         8 EZEKIEL HOUGHTON        5            1641      5.03
## 9         9 STEFANO LEE            5            1411      2.29
## 10        10 ANVIT RAO           5            1365      1.94
## # i 31 more rows
## # i 1 more variable: Score_minus_Expected <dbl>
```

Calculate five most overperformed relative to their expected score

```
overperform_df<-final_df%>%
  arrange(desc(Score_minus_Expected))
head(overperform_df,n=5)
```

```
## # A tibble: 5 x 6
##   Pair_Num Player_name      Total_Points Player_pre_Rating ELO_total
##   <dbl>   <chr>          <dbl>          <dbl>      <dbl>
## 1         3 ADITYA BAJAJ           6            1384      1.95
## 2        15 ZACHARY JAMES HOUGHTON  4.5            1220      1.37
## 3        10 ANVIT RAO             5            1365      1.94
## 4        46 JACOB ALEXANDER LAVALLEY  3              377      0.0432
## 5         9 STEFANO LEE            5            1411      2.29
## # i 1 more variable: Score_minus_Expected <dbl>
```

Calculate five most underperformed relative to their expected score

```
underperform_df<-final_df%>%  
  arrange(Score_minus_Expected)  
head(underperform_df,n=5)
```

```
## # A tibble: 5 x 6  
##   Pair_Num Player_name      Total_Points Player_pre_Rating ELO_total  
##   <dbl> <chr>              <dbl>          <dbl>      <dbl>  
## 1      25 LOREN SCHWIEBERT      3.5          1745      6.28  
## 2      30 GEORGE AVERY JONES      3.5          1522      6.02  
## 3      42 JARED GE              3            1332      5.01  
## 4      31 RISHI SHETTY          3.5          1494      5.09  
## 5      35 JOSHUA DAVID LEE      3.5          1438      4.96  
## # i 1 more variable: Score_minus_Expected <dbl>
```

Conclusion

Based on the overperformed relative to their expected score data, Aditya Bajaj scored 6.0 total points even though his predicted score was only 1.95 points. This means Aditya performed much better than expected. Similarly, Zachary James Houghton and Anvit Rao also performed significantly better than the model predicted. Based on the underperformed relative to their expected score data, Loren Schwiebert scored only 3.5 total points even though his predicted score was 6.28 points. This means he performed much worse than expected. Similarly, George Avery Jones, Jared Ge, Rishi Shetty, and Joshua David Lee also scored fewer points than the model predicted.

Citation

[Large language model]. google.com. Accessed October 4, 2026. <https://share.google/aimode/j7NEiRnQ1M8CZXzpC>